

Building Knowledge Graphs: From Raw Data to Structured Knowledge

Concepts, Construction Pipelines, and Practical Refinement

Andy Liang (zhl241@ucsd.edu),
Subhasis Dasgupta(sudasgupta@ucsd.edu).

San Diego Supercomputer Center
UC San Diego

What is a Knowledge Graph?

Definition

A knowledge graph represents entities as nodes and typed relationships as edges.

When does a graph become a Knowledge Graph?

A graph becomes a knowledge graph when its nodes and relationships are organized using background knowledge such as an ontology, schema, constraints, or rules.

Without this semantic layer, it is only a **data graph**.

From Data Graph to Knowledge Graph

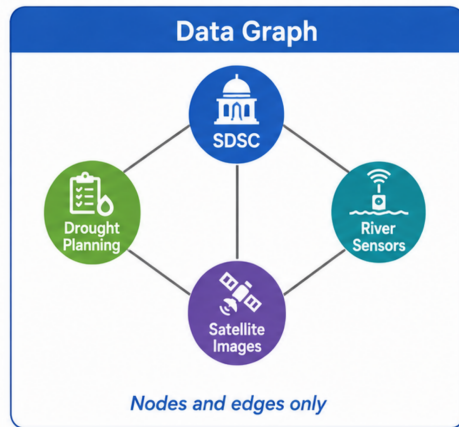
Running Example

SDSC supports California's drought planning using satellite images and river sensors.

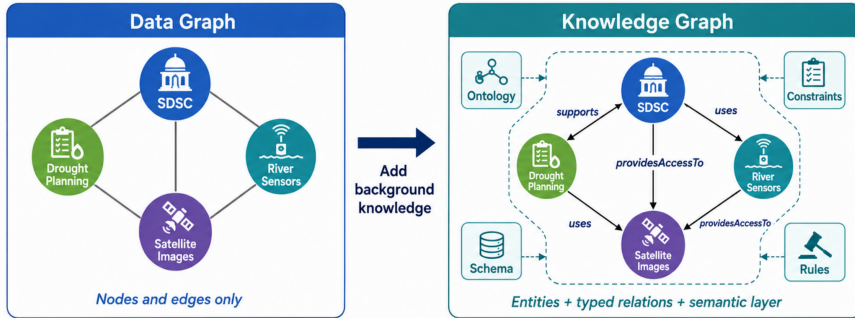
The Center for Satellite Imaging provides SDSC with access to these sensors for this workshop.

From Text to Graph

The text identifies entities and connections that can first be represented as a **data graph**.



Data Graph vs. Knowledge Graph



A graph becomes a knowledge graph when **entities and relations** are organized using background knowledge such as an **ontology, schema, constraints, or rules**.

Without this semantic layer, it is only a data graph.

A data graph captures connections; a knowledge graph adds typed relations, ontology, constraints, and semantic meaning.

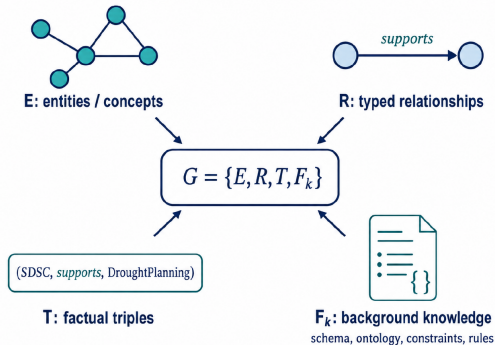
Formal Definition of a Knowledge Graph

$$G = \{E, R, T, F_k\}$$

- E : entities or concepts
- R : typed relationships
- T : factual triples
- F_k : background knowledge that constrains facts

Central idea

The background knowledge F_k gives semantic meaning to the entities, relationships, and facts.



Formal Definitions: Knowledge Graph and Construction

Definition 2: Knowledge Graph Construction

Knowledge graph construction is a mapping from a data source and its background knowledge into a knowledge graph:

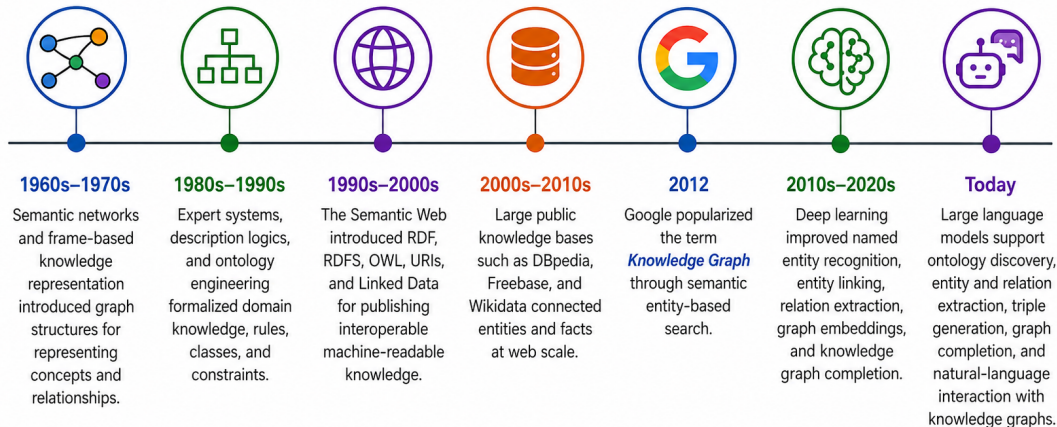
$$f : D \times f_k(D) \rightarrow G$$

where D is the set of data sources and $f_k(D)$ represents domain or background knowledge about the target data.

Main implication

Knowledge graph construction is not simply triple extraction: background knowledge—such as rules, schemas, ontologies, or learned representations—is required to organize and constrain the facts.

Knowledge Graphs: Concept and Historical Evolution



Basic Idea of Knowledge Graph Construction

1. Data Sources

Text and documents
Tables and databases
APIs
Existing knowledge graphs

2. AI/ML Extraction

Entity recognition
Entity typing and linking
Relation extraction
LLM-assisted extraction

3. Semantic Construction

Ontology alignment
Triple generation
Validation and provenance
Reasoning

Raw Data $\Rightarrow$ **Candidate Facts** $\Rightarrow$ **Validated Knowledge**

Key message

AI/ML proposes entities and relations; ontology and validation determine what becomes part of the knowledge graph.

Text Extraction

Simple Extraction vs. **Ontology-Grounded Extraction**

From unstructured text to entities, relations, and validated triples

Simple Knowledge Extraction from Text

Goal

Transform unstructured text into entities and relationships that can be represented as knowledge triples.

Text $\implies$ Entities $\implies$ Relations $\implies$ Triples

Entity Processing

- Named Entity Recognition
- Entity Typing
- Entity Linking
- Coreference Resolution

Relation Processing

- Relation Extraction
- Subject–Predicate–Object identification
- Triple generation
- Raw graph construction

Ontology-Grounded Knowledge Extraction from Text

Goal

Use an ontology to guide extraction so that generated facts are consistent, interpretable, and reusable.

Text $\implies$ Candidate Entities and Relations $\implies$ Ontology Alignment $\implies$ Validated Triples

Ontology Guidance

- define classes and properties;
- specify domain and range;
- reuse canonical identifiers;
- align relations with Wikidata or another ontology.

Controlled Construction

- extract ontology-relevant facts;
- replace synonymous relations;
- validate ontology conformance;
- reduce unsupported or hallucinated triples.

Structural Interpretation



Entity and Relation Extraction



Coreference Resolution



Statistical and Deep-Learning Methods

Example Text

SDSC supports California's drought planning using satellite images and river sensors. The Center for Satellite Imaging provides SDSC with access to these sensors for this workshop.

Sentence 1

- Subject: SDSC
- Action: supports
- Object: drought planning
- Resources: images and sensors

Sentence 2

- Subject: Imaging Center
- Action: provides access
- Recipient: SDSC
- Context: workshop

Entity and Relation Extraction

Entities

- SDSC: Organization
- California: Location
- Drought planning: Activity
- Satellite images: Data
- River sensors: Instrument
- Imaging Center: Organization
- Workshop: Event

Candidate Relations

(SDSC, supports, DroughtPlanning)

(DroughtPlanning, uses, SatelliteImages)

(DroughtPlanning, uses, RiverSensors)

(ImagingCenter, providesAccess, SDSC)

Coreference Resolution

Mentions referring to earlier concepts

- **these sensors** → river sensors
- **this workshop** → the current workshop event
- **the Center** → Center for Satellite Imaging

Resolved Knowledge

(ImagingCenter, providesAccessTo, SDSC)

(Access, appliesTo, RiverSensors)

(Access, forEvent, Workshop)

Statistic-Based Coreference Resolution

Clustering

Group similar mentions:

{Center, the center}

Decision Tree

Use type and proximity:

these sensors $\rightarrow$ river sensors

Mention Scoring

Rank antecedents:

it $\rightarrow$ SDSC

Limitation

These methods depend heavily on engineered features and may struggle with long-distance or ambiguous references.

Source: Zhong et al., *A Comprehensive Survey on Automatic Knowledge Graph Construction*, ACM Computing Surveys, 2023.

Deep-Learning Coreference Resolution

CNN

Uses local context:

the center → Imaging Center

RNN/LSTM

Uses cross-sentence context:

it → SDSC

Attention

Scores mention–antecedent pairs:

these sensors → river sensors

Standard process

Mention detection $\implies$ antecedent scoring $\implies$ coreference clustering

Agentic Knowledge Graph Construction

Documents $\Rightarrow$ Extract $\Rightarrow$ Align $\Rightarrow$ Validate $\Rightarrow$ KG

Extraction Agent

Entities
Relations
Candidate triples

Schema Agent

Ontology matching
Canonicalization
Schema expansion

Validation Agent

Source checking
Constraint checking
Reflection and repair

Recent direction

Agentic pipelines iteratively construct, inspect, and repair the knowledge graph instead of accepting one-pass LLM output.

Examples: AutoSchemaKG (2025), AGENTiGraph (2025), and FinReflectKG (2025). Orchestration may be implemented using frameworks such as LangChain or LangGraph.

Challenges of LLM and Agentic KG Construction

Large-Model Problems

- hallucinated triples
- inconsistent predicates
- missing facts
- weak schema conformance
- nondeterministic output

Agentic-System Problems

- errors propagate across agents
- repeated calls increase cost
- workflows add latency
- stopping criteria are unclear
- validation remains difficult

Practical implication

Use ontology constraints, source-grounded validation, provenance, confidence scores, and human review before adding generated facts to the knowledge graph.

Sources: Zhong et al. (2023); Mihindukulasooriya et al. (2023); Feng et al. (2024); Kommineni et al. (2024); Ghanem et al. (2025).

How Widely Are Agentic Systems Used?

Enterprise Adoption

- 62% experimenting with agents
- 23% scaling agentic systems
- 52% report active agent use
- 79% report some adoption

Production Reality

- 68% stop within 10 steps
- 70% use prompted base models
- 74% depend on human evaluation
- Reliability remains the main concern

Interpretation

Agentic AI is widely piloted, but autonomous, production-scale use remains limited and strongly dependent on human validation.

Sources: McKinsey (2025); PwC AI Agent Survey (2025); Google Cloud Agent Study (2025); Pan et al., *Measuring Agents in Production* (2025).

Agentic Systems for Knowledge Extraction

KARMA: Multi-Agent Extraction

- 9 specialized agents
- 1,200 PubMed articles
- 38,230 new entities
- 83.1% verified correctness
- 18.6% fewer conflict edges

Tree-KG: Iterative Expansion

- LLM-guided graph construction
- iterative knowledge expansion
- F1 score: 0.81
- 12–16% above second-best

Takeaway

Agentic extraction is promising because specialized agents can extract, verify, align, and repair knowledge before graph insertion.

Raw KG $\implies$ Completion $\implies$ Alignment and Fusion $\implies$ Refined KG

Knowledge Completion

- fill missing triple elements
- derive new supported facts
- assess triple correctness

Knowledge Fusion

- align equivalent attributes
- match duplicate entities
- merge facts from multiple KGs

Purpose

Extraction builds a raw graph; refinement repairs, enriches, and integrates it.

Missing Triple Completion

Semantic and Ontology-Aware Completion

Coreference-Supported Completion

Completed and Validated Knowledge

What is Knowledge Graph Completion?

Definition

Knowledge graph completion fills missing elements in incomplete triples and derives new triples from completed knowledge.

$$(h, r, ?) \quad (?, r, t) \quad (h, ?, t)$$

Running Example: Existing Facts

(SDSC, supports, DroughtPlanning)

(DroughtPlanning, uses, RiverSensors)

Completion Question

Can the graph infer or retrieve the missing fact?

Semantic and Ontology-Aware Completion

Semantic Completion

Use embeddings, neighborhoods, or textual meaning to predict a missing fact.

(DroughtPlanning, uses, ?)

$\Rightarrow$ SatelliteImages

Ontology-Aware Completion

Use type, hierarchy, domain, and range constraints.

$\text{uses} : \text{Activity} \rightarrow \text{DataOrInstrument}$

Reject:

$\text{uses}(\text{DroughtPlanning}, \text{California})$

Difference

Semantic evidence ranks plausible answers; ontology constraints determine which answers are valid.

Coreference-Supported Completion

Text

The Center for Satellite Imaging provides SDSC with access to these sensors for this workshop. The center also shares satellite imagery with it.

Resolve Mentions

the center → Imaging Center

it → SDSC

these sensors → River Sensors

Complete the Graph

(ImagingCenter, shares, SatelliteImagery)

(ImagingCenter, sharesWith, SDSC)

Key idea

Resolving repeated mentions prevents duplicate nodes and recovers facts distributed across sentences.

Knowledge Alignment

Entities, relations, attributes, time, and restrictions

Ambiguity Resolution

Canonical entities and ontology-guided relations

Entity and Relation Alignment

KG 1

SDSC

riverSensor

uses

KG 2

SanDiegoSupercomputerCenter

streamGauge

employsInstrument

$\text{SDSC} \equiv \text{SanDiegoSupercomputerCenter}$

$\text{uses} \approx \text{employsInstrument}$

Additional Alignment Dimensions

Attribute Alignment

$\text{sensorID} \equiv \text{instrumentIdentifier}$

Temporal Alignment

$\text{Aug. 4, 2026} \equiv 2026-08-04$

Restriction Alignment

$\text{uses} : \text{Activity} \rightarrow \text{Instrument}$

$\text{employsInstrument} : \text{Project} \rightarrow \text{Sensor}$

Extracted Access Statement

The Center for Satellite Imaging provides SDSC with access to these sensors for this workshop.

Access Scope

- Recipient: SDSC
- Resource: river sensors
- Purpose: workshop

Access Restrictions

- Validity: workshop duration
- Admin domain: SDSC
- Network: approved SDSC access

`AccessGrant(SDSC, RiverSensors, Workshop)`

Entity Ambiguity Resolution

Running Text

The Center for Satellite Imaging provides SDSC with access to these sensors for this workshop. The center supports it during the event.

Ambiguous Mentions

the center

it

these sensors

Resolved Entities

the center → ImagingCenter

it → SDSC

these sensors → RiverSensors

Ontology-Guided Relation Resolution

Candidate Relations

- provides
- shares
- grants access
- owns

Ontology Constraints

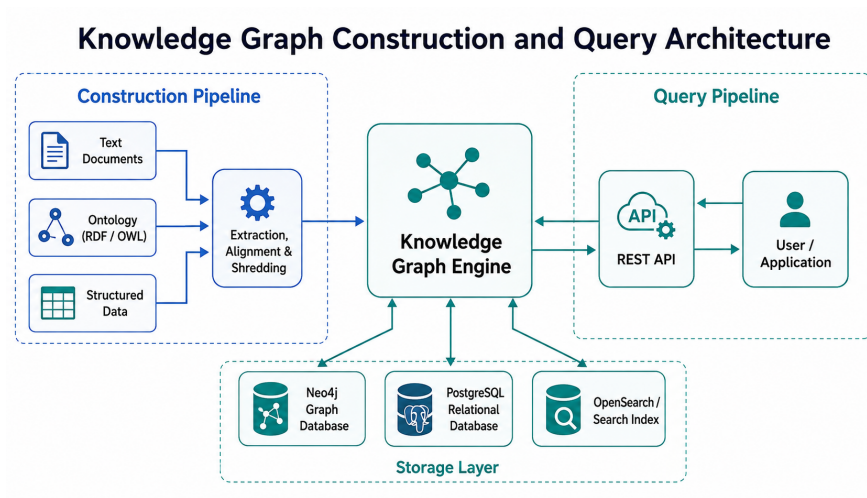
grantsAccess :

Organization $\times$ Organization $\times$ Resource

Resolved Relation

The argument types and sentence context select **grantsAccess**, not **owns** or **shares**.

Knowledge Graph Construction and Query Architecture



Construction pipeline, storage layer, and query access through REST API.

Exercise Setup: Get the Repository

Step 1: Clone the exercise repository

```
git clone https://github.com/sdsc/sdsc-summer-institute-2026.git  
cd sdsc-summer-institute-2026/4.1_knowledge_management/exercise
```

Expected folders

```
databases/ notebooks/ requirements.txt
```

Goal

Move into the exercise folder and confirm that both `databases` and `notebooks` are available.

Exercise Setup: Deploy the Databases

Step 2: Start the KG database services

```
cd databases  
ls  
bash launch_kg_db.sh
```

Expected result

```
Submitted batch job 52630400
```

Next step

After the job starts successfully, follow Section 2 of `README.md` and copy the generated environment variables.